

LAB_3

Linear Data Structures

Session Objectives

By the end of this session, you will be able to:

1. Implement and manipulate doubly linked lists
2. Combine stacks and queues to model some algorithms
3. Build priority queues using linked lists
4. Analyze the trade-offs between different linear structure implementations

Core Exercises

Exercise 1: Content Feed Navigation with Doubly Linked List

Objective: Implement a browsable content feed using a doubly linked list for efficient forward/backward navigation. Social media feeds allow users to scroll through posts, jump between related content, and navigate history. A doubly linked list provides natural bidirectional traversal. You're building a "Story Highlights" feature where users can navigate through a sequence of stories, moving both forward and backward.

Requirements:

1. Create a StoryNode structure containing:
 - story_id: unique identifier
 - user_id: author identifier
 - content_preview: string (title/first few words)
 - timestamp: time posted
 - views: integer counter
 - next: pointer to next story
 - prev: pointer to previous story
2. Implement a DoublyLinkedList with:
 - head: pointer to first node
 - tail: pointer to last node
 - current: pointer to currently viewed story
 - size: number of stories
3. Implement core operations:
 - add_story(node): add to end of feed (maintain chronological order)
 - remove_story(story_id): delete a specific story from feed
 - move_forward(): advance current to next story (return story data)
 - move_backward(): move current to previous story (return story data)
 - jump_to(story_id): set current to specific story ($O(n)$ search)
 - insert_after(current_id, new_story): insert after a specific story
 - display_around_current(k): show k stories before and after current
4. Implement engagement tracking:
 - track_view(current): increment view count for current story

- `most_viewed()`: return story with highest views (single traversal)
- `reorder_by_views()`: reorganize feed so most viewed stories appear first

Examples:

- Initial feed (chronological):
 - [Story1: "Morning coffee"] <-> [Story2: "Workout complete"] <-> [Story3: "Sunset photo"]
- After viewing Story2 twice and Story3 once:
 - Views: Story1(0), Story2(2), Story3(1)
- After `reorder_by_views()`:
 - [Story2: "Workout complete"] <-> [Story3: "Sunset photo"] <-> [Story1: "Morning coffee"]

Complexity Analysis Questions:

1. Compare time complexity of forward/backward navigation in singly vs. doubly linked lists
2. What is the time complexity of removing a node given only a pointer to that node?
3. How much extra memory per node does a doubly linked list require compared to singly linked?
4. In what scenarios would the bidirectional navigation justify the memory overhead?

Exercise 2: Activity Feed Processing with Stacks and Queues

Objective: Combine stack and queue structures to model different content consumption patterns. Social platforms process user activities through various patterns: recent activity stacks (LIFO) for "Recent Highlights" and pending notification queues (FIFO) for "Unread Alerts".

Part A: Recent Activity Stack

Users want to see their most recent interactions first. Implement a stack-based "Activity History".

Requirements:

1. Create an `ActivityStack` structure (using pointers) and implement the following operations:
 - `push(activity)`: add new activity to top (type: "like", "comment", "share", "follow")
 - `pop()`: remove and return most recent activity
 - `peek()`: view most recent without removing
 - `is_empty()`: check if stack is empty
 - `size()`: current stack size
 - `display_recent(n)`: show n most recent activities
2. Implement "undo" functionality:
 - `undo_last()`: revert the most recent action (e.g., unlike a post)
 - Maintain a separate `undo_stack` to track actions that can be redone

Part B: Notification Queue

Users receive notifications that should be processed in the order they arrived. Implement a queue-based "Notification System".

Requirements:

1. Create a `NotificationQueue` structure (using pointers) and implement the following operations:

- enqueue(notification): add notification to end of queue
 - dequeue(): process and remove oldest notification
 - front(): view next notification without removing
 - is_empty(): check if queue is empty
 - size(): current queue length
 - display_pending(): show all unread notifications in order
2. Implement priority boost:
 - priority_enqueue(notification): add to front for urgent notifications
 - Requires maintaining queue order integrity

Part C: Integrated Feed Processing

Combine both structures (using pointers) to model a complete activity processing system.

Requirements:

1. Create a FeedProcessor structure that maintains:
 - recent_activities: stack of user's own actions
 - notification_queue: queue of incoming notifications
 - processed_log: queue of already-processed items
2. Implement:
 - process_incoming(): move oldest notification from queue to recent stack
 - batch_process(k): process k oldest notifications
 - clear_history(): move all from recent stack to processed_log
 - get_stats(): return sizes of all three structures

Examples:

- Initial state:
 - Recent stack (top first): [Shared photo, Commented on post, Liked video]
 - Notification queue (front first): [New follower, New like, New comment]
- After process_incoming():
 - Dequeue "New follower" from notification queue
 - Push "New follower" to recent stack
- Result:
 - Recent stack: [New follower, Shared photo, Commented on post, Liked video]
 - Notification queue: [New like, New comment]

Complexity Analysis Questions:

1. Compare time complexity of all operations
2. For a queue, which implementation (array circular buffer vs. linked list) is more memory efficient for unknown size?
3. What is the amortized time complexity of a sequence of n enqueue/dequeue operations?
4. How would you implement a deque (double-ended queue) to support both stack and queue operations?

Exercise 3: Engagement-Based Priority Queue

Objective: Implement a priority queue using a sorted linked list to manage content by engagement metrics. Social networks rank content by relevance, urgency, or popularity. A priority queue allows efficient retrieval of the highest-priority item. You're

building a "Trending Now" feature that maintains posts sorted by current engagement score.

Requirements:

1. Create a Post structure containing:
 - o post_id: unique identifier
 - o user_id: author
 - o content: string
 - o timestamp: time posted
 - o likes: integer
 - o comments: integer
 - o shares: integer
 - o engagement_score: computed as $(likes \times 1 + comments \times 2 + shares \times 3)$
2. Implement a PriorityQueue structure using a sorted singly linked list:
 - o Maintain list in descending order of engagement_score
 - o head: pointer to highest priority post
3. Core operations:
 - o enqueue(post): insert post maintaining sorted order ($O(n)$)
 - o dequeue_max(): remove and return highest priority post ($O(1)$)
 - o peek_max(): view highest priority without removing ($O(1)$)
 - o is_empty(): check if queue is empty
 - o size(): return number of posts
4. Priority updates:
 - o update_score(post_id, new_likes, new_comments, new_shares):
 - Recalculate engagement score
 - Remove post and re-insert in correct position (maintain sorted order)
 - o refresh_all(): recalculate all scores and re-sort the entire queue
5. Trending window:
 - o get_top_k(k): return k highest priority posts
 - o decay_older_than(timestamp): reduce scores of posts older than given time (simulate time decay)

Examples:

- Initial queue (highest first):
 - [Post3: score 95] -> [Post1: score 82] -> [Post2: score 47] -> [Post4: score 23]
- After Post2 gets 10 new likes (score increases from 47 to 57):
 - update_score(Post2, +10 likes)
 - Result: [Post3:95] -> [Post1:82] -> [Post2:57] -> [Post4:23]
- After decay_older_than(1 hour) reduces Post3 by 20%:
 - [Post1:82] -> [Post3:76] -> [Post2:57] -> [Post4:23]
- Bonus Challenge:
 - Implement a dual-priority queue that maintains two separate queues:
 - o viral_queue: posts with engagement_score > 100
 - o normal_queue: all other posts

When dequeuing, return from viral_queue with probability 0.7 (70% chance) to boost viral content visibility.

Complexity Analysis Questions:

1. Compare time complexity of enqueue for:
 - o Sorted linked list implementation
 - o Unsorted linked list with find-max each time

2. What is the worst-case time complexity of `update_score`? How could you optimize it?
3. For a social network with millions of posts, would a sorted linked list be practical? Why or why not?
4. Space complexity: How much extra memory per node does a priority queue require compared to a standard linked list?
5. If engagement scores update frequently (every few seconds), what data structure would be more efficient than a sorted linked list?

Final Question

Based on your previous work, propose how you might combine in your future semester project:

- Doubly linked lists for feed navigation
- Stacks/queues for activity processing
- Priority queues for content ranking

How would these structures work together to build a complete social media feature (e.g., "Trending Feed with Friend Activity")?

General Instructions

Repository Setup

Create branch: LAB_03_LinearDS

File Structure:

Follow the same structure of LAB_02

Git Requirements:

Same as for LAB_02

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 10 • Each Team (of number BB) Upload at the end of the session a very short PDF document and name it TBB_LAB3_AES.pdf, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	Note: 10 • You finish all the implementation of all exercises, • You finalize the previous report and give it a new name TBB_LAB3_BFM.pdf and upload it before Friday midnight of the same week, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.